SOFTWARE DEVELOPER PORTFOLIO

Software Requirements Specification (SRS)
+ AI Design & Development Prompts

Document information	Value
Project	Personal Software Developer Portfolio
Prepared for	Ashlin Siddharth
Document version	1.0 – Initial baseline
Date	8 November 2026
Development environment	Google Antigravity
Source control	GitHub (connected to Antigravity)
Proposed hosting	GitHub Pages
Status	Draft for review and confirmation

Purpose: define the portfolio before UI design and implementation.

Document Control

Version	Date	Description	Status
1.0	08-Nov-2026	Initial SRS and prompt pack	Draft

Contents

1. Introduction

2. Overall Description

3. User and Portfolio Content

4. Functional Requirements

5. Non-Functional Requirements

6. UI/UX and Design Requirements

7. Technical Architecture

8. GitHub and Deployment Requirements

9. Testing and Acceptance Criteria

10. Assumptions, Constraints and Open Decisions

11. Claude Prompt Pack

12. Antigravity Prompt Pack

13. Final Content Checklist

1. Introduction

1.1 Purpose

This Software Requirements Specification defines the scope, content, behaviour, design expectations, technical approach, quality requirements and deployment process for a personal software developer portfolio website for Ashlin Siddharth. It is intended to be supplied to Claude for requirements review and UI/UX planning, and then used as the implementation baseline in Google Antigravity.

1.2 Project Scope

The product is a responsive, accessible, professional portfolio website that presents the owner's profile, technical capabilities, selected projects, education, experience or leadership activities, resume and contact channels. It should make it easy for recruiters, hiring managers, collaborators and peers to understand the owner's work and reach out.

The initial release is a primarily static website. A database, authentication system, admin dashboard and custom backend are out of scope unless a later requirement justifies them. Contact functionality must use a reliable, explicitly configured method; it must not pretend to submit a message if no delivery service is configured.

1.3 Intended Audience

Recruiters and hiring managers evaluating skills and projects.

Developers and potential collaborators reviewing code and technical work.

University faculty, peers and professional contacts.

The portfolio owner, who will maintain project details and links.

1.4 Definitions

Term	Meaning
SRS	Software Requirements Specification.
CTA	Call to action, such as View Projects or Contact Me.
Responsive	Layout adapts to mobile, tablet and desktop screen sizes.
GitHub Pages	Static website hosting provided through a GitHub repository.
Antigravity	AI-assisted development environment used to create and modify the project.

2. Overall Description

2.1 Product Perspective

The portfolio is an independent personal website. It will be developed locally in Google Antigravity, tracked in a GitHub repository and published as a static site through GitHub Pages. The deployed site must work without requiring the visitor to install software or sign in.

2.2 Product Goals

Establish a credible professional identity as a software developer and BCA Data Science student.

Showcase a small number of strong projects with clear problem statements, contributions, technology and outcomes.

Provide direct access to GitHub, LinkedIn and a downloadable resume.

Deliver a polished, fast, mobile-friendly and accessible experience.

Allow the owner to update content and add projects without restructuring the entire application.

2.3 User Classes

User class	Needs
Recruiter / hiring manager	Quick overview, skills, project evidence, resume and contact details.
Technical reviewer	Project architecture, technologies, GitHub repositories, implementation details.
Collaborator	Areas of interest, project links and a clear contact route.
Portfolio owner	Simple content structure and maintainable code.

2.4 Operating Environment

Modern desktop and mobile browsers: Chrome, Edge, Firefox and Safari.

Responsive viewport support from small mobile screens to large desktop displays.

Static hosting on GitHub Pages over HTTPS.

Development and Git operations performed through Google Antigravity and a connected GitHub account.

3. User and Portfolio Content

3.1 Confirmed Profile Information

Field	Initial value
Full name	Ashlin Siddharth
Professional title	Software Developer (positioning statement; refine as desired)
Education	BCA – Data Science
Institution	Presidency University, Bengaluru
Expected graduation	2027
Location	Bengaluru, India
LinkedIn	https://www.linkedin.com/in/ashlin-siddharth-191861165
GitHub	https://github.com/ashlinsiddharth

3.2 Proposed Content Sections

Section	Required content
Home / Hero	Name, concise developer headline, short value proposition, primary CTAs and optional profile visual.
About	Short biography, interests, approach to problem-solving and current learning focus.
Skills	Grouped languages, frameworks/libraries, databases, tools and development concepts. Only verified skills.

Section	Required content
Projects	Project cards and detailed project information: problem, solution, role, stack, features, status and links.
Experience / Activities	Internships, practical work, leadership, event responsibilities or relevant activities, with dates and factual descriptions.
Education	Degree, university, expected graduation and relevant academic work.
Resume	Clearly labelled resume download, with a valid file and accessible link.
Contact	Email and professional links; optional form only after a working delivery mechanism is configured.

3.3 Candidate Projects for Review

These are candidate entries based on previously discussed work. They are not approved portfolio copy. Before publication, confirm public visibility, project status, personal contribution, technology stack and repository/demo URLs.

Candidate	Potential portfolio angle	Publication checks
RAG-Based Intelligent Academic Information Assistant for University Students	Academic assistant using document ingestion, retrieval and generated answers; FAISS and Sentence Transformers were part of the planned/recorded approach.	Confirm completed features, final architecture, individual contribution, repository and demo.
VeriSync – Secure Assessment Portal	Secure internship assessment portal; present the user's actual development ownership and implemented modules.	Confirm permission to publish, confidentiality constraints, screenshots and public code availability.
INPM Trust Website	Website design and implementation for a trust, including public pages and admin/contact workflows.	Confirm ownership, live URL, technologies and which modules were personally implemented.
Other projects	Add only projects with demonstrable contribution and accurate descriptions.	Confirm title, problem, role, stack, links and outcome.

3.4 Content Integrity

Do not invent employment, internships, certifications, awards, metrics, client names or project outcomes.

Do not claim proficiency in a language or framework without owner confirmation.

Distinguish completed features from planned, experimental or in-progress features.

Use concise, specific writing and explain the owner's contribution to group projects.

Use placeholders for missing information during development; do not publish visible placeholder text.

4. Functional Requirements

ID	Feature	Requirement
FR-01	Navigation	The site shall provide clear navigation to all primary sections or pages.
FR-02	Hero section	The home view shall identify the owner and communicate the developer positioning with clear next actions.
FR-03	About	The site shall present a concise biography and current academic/professional focus.

ID	Feature	Requirement
FR-04	Skills	The site shall group verified skills into understandable categories.
FR-05	Projects	The site shall display project summaries and allow visitors to open project details, repositories or demos where available.
FR-06	Project detail	Each project detail shall include problem/context, solution, role/contribution, stack, features and links where known.
FR-07	Education	The site shall show the owner's degree, institution and expected graduation year.
FR-08	Experience/activities	The site shall support factual work, internship, leadership or activity entries.
FR-09	Resume	The site shall provide a working resume download or clearly labelled resume link.
FR-10	Contact	The site shall expose working email and professional profile links.
FR-11	Contact form	If a form is implemented, it shall validate fields and provide truthful success/error feedback through a configured delivery service.
FR-12	Responsive layout	The site shall adapt to mobile, tablet and desktop layouts.
FR-13	External links	External links shall use correct destinations and appropriate safe link behaviour.
FR-14	Footer	The footer shall include identity, useful navigation and relevant social/contact links.
FR-15	Maintainability	Content shall be organised so projects, skills and experience can be updated without editing unrelated layout components.

5. Non-Functional Requirements

ID	Quality attribute
NFR-01 Performance	Optimise images, avoid unnecessary dependencies and keep initial page load fast. Set a target of good Core Web Vitals; validate with Lighthouse.
NFR-02 Responsive design	No horizontal overflow at common mobile widths; layouts remain usable on tablet and desktop.
NFR-03 Accessibility	Semantic HTML, keyboard-accessible navigation, visible focus states, sufficient contrast, descriptive alt text and labelled form fields.
NFR-04 Compatibility	Support current stable versions of major modern browsers.
NFR-05 Security	Do not commit secrets, API keys, private contact data or credentials. Use HTTPS and avoid unsafe HTML injection.
NFR-06 Reliability	All navigation, downloads and external links must work; no dead buttons or fake interactions.
NFR-07 SEO	Provide page title, meta description, favicon, semantic headings, social sharing metadata and a sensible URL structure.

ID	Quality attribute
NFR-08 Maintainability	Use reusable components, consistent naming, clear folder structure and documented setup/deployment steps.
NFR-09 Visual quality	Consistent spacing, typography, colour tokens, interaction states and responsive behaviour.
NFR-10 Privacy	Collect only necessary visitor information; disclose any form provider or analytics if added.

6. UI/UX and Design Requirements

6.1 Design Direction

Create a distinctive, polished developer portfolio with a modern editorial/technical feel. The design should feel confident and personal, not like a generic template or a crowded dashboard. The exact visual direction, colour palette and typography are to be proposed during the design phase and approved before implementation.

6.2 Design Principles

Strong visual hierarchy: the owner's identity and best work should be immediately discoverable.

Readable typography and generous spacing; avoid oversized headings that dominate the content.

Use restrained motion and transitions; respect prefers-reduced-motion.

Use project screenshots or diagrams only when they are real, relevant and permitted.

Keep navigation predictable and mobile navigation easy to operate.

Design empty, hover, focus, active and error states rather than only the ideal desktop view.

6.3 Proposed Information Architecture

Route / section	Purpose
/	Home, hero and overview
#about	About the developer
#skills	Technical skills
#projects	Selected projects
#experience	Experience and activities
#education	Education
#contact	Contact and social links
/resume.pdf	Resume asset, if supplied

The design phase may recommend a single-page layout or a small multi-page structure. Keep the navigation and URLs simple.

7. Technical Architecture

7.1 Proposed Stack (subject to design-phase confirmation)

Layer	Proposed choice	Reason
Framework	React + Vite	Suitable for a fast static portfolio and straightforward GitHub Pages deployment.

Layer	Proposed choice	Reason
Language	TypeScript	Improves maintainability and catches common errors.
Styling	Tailwind CSS or CSS Modules	Consistent responsive styling; choose one approach and avoid unnecessary mixing.
Icons	One consistent icon library or inline SVG	Consistent visual language and lightweight assets.
Hosting	GitHub Pages	Static hosting integrated with the GitHub repository.
CI/CD	GitHub Actions	Build and publish on approved pushes to the deployment branch.

7.2 Architecture Guidelines

Use reusable components for navigation, buttons, section headings, project cards and footer.

Separate content data from presentation where practical (for example, project and skill data arrays).

Use a clear src/components, src/sections or src/pages, src/data, src/assets structure as appropriate.

Use relative asset paths or configure the Vite base path correctly for the GitHub Pages repository URL.

Do not add a backend unless a requirement cannot be met safely with static hosting.

If a contact form requires a third-party service, obtain the owner's approval and configure secrets safely.

8. GitHub and Deployment Requirements

Create or use a dedicated GitHub repository with a clear name such as ashlin-portfolio.

Connect the repository to Google Antigravity and verify the correct GitHub account and repository before making changes.

Use meaningful commits and keep the main branch deployable.

Configure the framework build command and output directory correctly.

Use GitHub Actions to build and deploy the site to GitHub Pages.

Configure repository Settings → Pages to use the GitHub Actions deployment source.

Verify the deployed URL, asset paths, refresh behaviour, mobile layout and browser console after deployment.

Never commit .env files, access tokens, private keys or credentials.

Document how to run locally, build, preview and deploy.

Deployment note: GitHub Pages is static hosting. A client-side contact form does not automatically send email; a form provider or backend endpoint must be configured and tested before claiming that the form works.

9. Testing and Acceptance Criteria

Test ID	Acceptance criterion	Verification
AT-01	All primary navigation items reach the correct section/page.	Manual navigation test.
AT-02	Hero clearly identifies Ashlin and provides working primary actions.	Content and link review.
AT-03	All published projects have accurate descriptions and working links.	Owner review and link test.

Test ID	Acceptance criterion	Verification
AT-04	Resume link downloads or opens the correct supplied file.	Browser test.
AT-05	Email, LinkedIn and GitHub links open the intended destinations.	Link test.
AT-06	No horizontal overflow or clipped content on mobile.	Responsive browser test.
AT-07	Keyboard users can navigate and see focus indicators.	Keyboard-only test.
AT-08	No dead buttons, broken images or visible placeholder content.	Manual QA.
AT-09	Production build completes without errors.	Build command / CI result.
AT-10	GitHub Pages deployment loads over HTTPS with all assets.	Live deployment test.
AT-11	No secrets or private credentials are present in repository history/current files.	Repository review.
AT-12	Lighthouse review is completed and major accessibility/performance issues are addressed.	Lighthouse audit.

10. Assumptions, Constraints and Open Decisions

10.1 Assumptions

The owner will provide final profile copy, skills, project details, resume and any approved images.

The first release is a static portfolio and does not need user accounts or an admin dashboard.

The owner has or will connect a GitHub account to Antigravity.

Project screenshots, logos and third-party assets will be used only when the owner has permission.

10.2 Constraints

GitHub Pages serves static content; dynamic server-side features require an external service or separate backend.

All published claims must be accurate and approved by the owner.

Antigravity-generated code must be reviewed, run and tested; AI output is not accepted as proof of correctness.

The live site must not expose secrets, private assessment details or confidential project information.

10.3 Decisions to Confirm Before Visual Design

Decision	Current baseline / decision needed
Visual theme	Ask owner to choose or approve a proposed light, dark or hybrid direction.
Site structure	Single-page scrolling portfolio or multiple pages; design phase to recommend.
Technology	React + Vite + TypeScript + Tailwind is proposed, not yet locked.
Contact method	Email link initially; form only after service/provider is selected.
Resume	Owner to supply final PDF.
Skills	Owner to confirm actual languages, frameworks, databases and tools.

Decision	Current baseline / decision needed
Projects	Confirm which candidate projects can be publicly showcased and provide links.
Profile photo	Optional; owner to supply if desired.
Custom domain	Optional future enhancement; GitHub Pages default URL is acceptable for first release.

11. Claude Prompt Pack

Use these prompts in order. Attach this SRS document to Claude before running Prompt 1.

Prompt 1 – SRS review and requirements validation

You are a senior Business Analyst and Software Architect. Review the attached Software Requirements Specification for Ashlin Siddharth's Software Developer Portfolio.

Tasks:
1. Identify ambiguity, contradictions, missing requirements and unnecessary complexity.
2. Separate confirmed facts from assumptions and proposed decisions.
3. Check whether the requirements are realistic for a static portfolio hosted on GitHub Pages.
4. Recommend a suitable frontend stack, but explain trade-offs before treating it as final.
5. Produce a prioritised requirement list (Must / Should / Could).
6. Produce a traceability matrix linking each functional requirement to a page/component and acceptance test.
7. List the exact information Ashlin must provide before final content is published.
8. Do not invent skills, work experience, certifications, project outcomes or technologies.
Return a revised SRS change list and questions for the owner. Do not write application code yet.

Prompt 2 – Information architecture and user journeys

Using the approved SRS, act as a senior UX architect. Create the portfolio information architecture and user journeys for:
- A recruiter evaluating Ashlin for a software developer opportunity.
- A technical reviewer exploring projects and GitHub repositories.
- A collaborator trying to contact Ashlin.

Recommend single-page versus multi-page navigation with reasons. Define page/section order, navigation labels, primary and secondary CTAs, content hierarchy, footer structure and mobile navigation behaviour. Keep the experience focused, fast and easy to scan. Do not invent personal content.

Prompt 3 – Visual design system and webpage specifications

Act as a senior product designer. Based on the approved SRS and information architecture, propose a distinctive professional developer-portfolio design system.

Deliver:
1. Two clearly differentiated visual directions with rationale.
2. A recommended colour palette with HEX values and semantic tokens.
3. Typography pairing and a responsive type scale.
4. Spacing, grid, container, border-radius, shadow and breakpoint rules.
5. Button, link, navigation, project-card, tag, timeline and form styles.
6. Motion principles, hover/focus/active states and reduced-motion behaviour.
7. Desktop, tablet and mobile layout specifications.
8. A section-by-section wireframe description for every page/section.
9. Accessibility and contrast considerations.
Do not generate code yet. Avoid generic template styling and visual clutter. Ask Ashlin to approve one direction before

proceeding.

Prompt 4 – Detailed webpage design handoff

Using the owner-approved visual direction, create a detailed UI handoff for the portfolio. For each page/section, specify:
- Exact content hierarchy and component breakdown.
- Desktop, tablet and mobile layout behaviour.
- Spacing, typography, colour and image treatment.
- Interaction states and animations.
- Empty, loading, error and success states wherever relevant.
- Accessibility notes.

Include a component inventory and design tokens. Use placeholders for any unconfirmed content and clearly label them. Do not fabricate copy or project details.

Prompt 5 – Design critique before development

Review the proposed portfolio design as a critical design lead. Evaluate clarity, originality, recruiter usability, mobile experience, accessibility, performance risks and consistency. Identify the 10 highest-impact improvements and revise the design specification. Confirm that the design can be implemented as a static GitHub Pages site without requiring a backend. Do not start coding.

Prompt 6 – Content and project case-study preparation

Help Ashlin prepare portfolio-ready content using only facts supplied in the SRS and subsequent owner responses. Create concise drafts for the hero, About, skills categories, education and project case studies. For each project, use: context/problem, objective, Ashlin's role, implementation, technologies, key features, current status, repository/demo links and learning/outcome. Mark missing facts as questions or [OWNER TO CONFIRM]. Never turn planned work into completed work.

12. Antigravity Prompt Pack

Run these prompts in Google Antigravity after the SRS and visual design have been approved. Keep the SRS and final design specification in the repository, for example under docs/.

Prompt A – Repository inspection and implementation plan

You are the lead frontend engineer for Ashlin Siddharth's portfolio. Read the approved SRS and UI design specification in the project. First inspect the existing repository and report its current structure, framework, scripts, dependencies and Git state. Do not overwrite or delete existing work.

Create an implementation plan mapped to the SRS requirements. Identify any missing assets or owner-provided content. Confirm the GitHub repository and current branch before any push. Do not begin implementation until you have presented the plan and any blocking questions.

Prompt B – Project scaffold and design tokens

Implement the approved technology stack and project structure only after the plan is approved. Establish global design tokens for colours, typography, spacing, containers, breakpoints, radii and shadows from the approved design specification. Configure routing or section navigation as specified. Create reusable layout primitives and verify the development server starts. Do not add unapproved features or invent content. Run the production build and report the result.

Prompt C – Build the page shell and navigation

Implement the responsive site shell, header/navigation, mobile menu, footer and page/section structure from the approved design. Match the approved design specification closely. Ensure semantic HTML, keyboard navigation, visible focus states, correct anchor offsets and accessible menu controls. Test mobile, tablet and desktop widths. Do not use dead buttons or placeholder links presented as working actions.

Prompt D – Build portfolio sections

Implement the approved Home/Hero, About, Skills, Projects, Experience/Activities, Education, Resume and Contact sections using reusable React components and structured content data. Use only owner-confirmed facts. Where content is missing, use clearly marked development placeholders that are easy to replace and must not appear in the production release. Ensure every CTA has a real destination or action.

Prompt E – Project case studies and assets

Implement project cards and project detail views according to the approved design. For every project, display only verified title, status, role, contribution, technology, features and links. Use supplied screenshots/assets only. Optimise image sizes, provide alt text and avoid remote assets that may disappear. Make repository/demo links open the correct URLs. Do not disclose confidential information.

Prompt F – Contact and resume functionality

Implement the contact and resume features exactly as approved. Prefer a direct mailto link unless a contact form provider has been explicitly configured. If a form provider is configured, use environment secrets only through the supported secure configuration; never hardcode API keys. Validate fields, provide accessible errors and truthful submission feedback. Verify the resume file exists and downloads/opens correctly. Do not simulate successful message delivery.

Prompt G – QA, accessibility and performance

Run a full QA pass against the SRS acceptance criteria. Check responsive layouts, navigation, links, resume download, images, keyboard operation, focus states, colour contrast, reduced motion, console errors and production build. Run available lint/type-check/test scripts. Identify and fix issues without changing the approved visual direction. Produce a checklist with pass/fail, evidence and remaining owner actions.

Prompt H – GitHub Pages configuration and deployment

Prepare this approved static portfolio for GitHub Pages. Inspect the framework configuration and set the correct base path for the actual repository name and deployment URL. Configure a GitHub Actions workflow to install dependencies, build the site and publish the correct output directory to GitHub Pages. Do not expose secrets. Before pushing, show the intended changed files, branch and commit summary. Ask for confirmation before any destructive Git action or force push. After deployment, verify the live URL, assets, navigation, refresh behaviour and mobile rendering. Report the exact deployment URL and any remaining issues.

Prompt I – Final release review

Act as a release engineer. Compare the deployed portfolio with the approved SRS and design specification. Verify every acceptance criterion, inspect the public site and repository for accidental secrets or private information, confirm all content is owner-approved, and list any incomplete items. Do not claim release readiness if a critical link, build, accessibility or deployment check fails. Produce a concise release report.

13. Final Content Checklist

Confirm professional headline and About biography.

Provide verified programming languages, frameworks, databases and tools.

Approve which projects can be publicly showcased.

Provide accurate project roles, current status, screenshots and repository/demo links.

Provide resume PDF and confirm it is current.

Confirm email address and preferred contact method.

Decide whether to include a profile photo.

Approve visual direction, colour palette and page structure.

Confirm GitHub repository name and public/private preference.

Review all copy and links before deployment.

End of Document

This document is the initial requirements baseline. Any change to scope, technology, content or deployment should be reviewed against the acceptance criteria and approved by the portfolio owner before implementation.